

Ergodicity Library: Integrated stochastic-process simulation, ergodicity diagnostics, and agent-based experimentation in Python

10 February 2026

Summary

`ergodicity_library` is an open-source Python package for research workflows centered on stochastic dynamics, especially settings where time-average behavior and ensemble-average behavior can diverge (Peters and Gell-Mann 2016; Peters 2019). The package integrates three layers that are often handled separately in ad hoc scripts: process simulation, diagnostics/analysis tools, and agent-based decision experiments.

In practice, many projects require repeated transitions between model definition, trajectory generation, fitting/diagnostic checks, and decision evaluation. The core contribution of this software is to make these transitions operationally coherent in one environment, reducing glue-code overhead and improving reproducibility (Kendiukhov 2025, 2024).

The package supports both baseline and advanced use-cases: Brownian and related processes, heavy-tailed families (including Lévy-stable variants), multiplicative growth settings, memory-dependent dynamics, and selected stochastic field simulations. This functionality is designed for researchers who need to compare assumptions quickly and inspect consequences with consistent interfaces.

Statement of need

Research questions in quantitative finance, economics, physics-inspired modeling, and computational social science frequently involve nontrivial stochastic assumptions. Typical workflows are fragmented: one code path for simulation, another for diagnostics, another for fitting, and often a separate implementation for decision logic. This fragmentation increases implementation time and makes it harder to compare model variants under consistent conditions.

The specific need addressed by `ergodicity_library` is an integrated software

framework for studies where process assumptions, transient diagnostics, and downstream decisions should be tested together. The intended users include applied researchers, graduate-level users, and research software practitioners who need to:

- simulate heterogeneous process families using a shared API,
- compare additive versus multiplicative dynamics,
- evaluate Gaussian versus heavy-tailed assumptions,
- inspect transient/preasymptotic behavior before asymptotic claims,
- couple stochastic dynamics to utility/portfolio/agent rules.

This package is not framed as a new mathematical theory. It is a research software contribution that operationalizes existing theory and methods into reusable computational workflows.

State of the field

The scientific Python ecosystem provides high-quality numerical building blocks, notably NumPy, SciPy, SymPy, and Matplotlib (Harris et al. 2020; Virtanen et al. 2020; Meurer et al. 2017; Hunter 2007). These tools are foundational and intentionally general.

For stochastic simulation specifically, packages such as `stochastic`, `sdeint`, and `sdepy` provide useful focused capabilities (stochastic developers 2026; sdeint developers 2026; sdepy developers 2026). They are valuable for generating paths or integrating SDEs, but generally do not aim to unify process abstraction, ergodicity-oriented diagnostics, fitting workflows, and agent-based experimentation under one research workflow model.

`ergodicity_library` was developed as a standalone package rather than a thin wrapper because its primary use-case depends on this cross-layer integration. The project targets a gap between low-level numerical primitives and narrowly scoped simulation utilities: comparative stochastic research where model choice, diagnostics, and decisions must be iterated jointly.

Software design

The software is organized into three primary modules:

- **Processes:** Ito and non-Ito process classes, including heavy-tailed, multiplicative, and memory-dependent variants.
- **Tools:** symbolic/numerical helpers for fitting, diagnostics, preasymptotic analysis, automation, and selected partial stochastic differential equation experiments.
- **Agents:** utility-based decision logic, portfolio/pool experiments, and optimization-oriented workflows.

A central design choice is a common process abstraction that allows users to swap process families without rewriting surrounding diagnostics and experiment code. This improves comparability across assumptions and lowers the cost of model stress testing.

Figure-based examples below illustrate the workflow span.

Heavy-tailed spread geometry

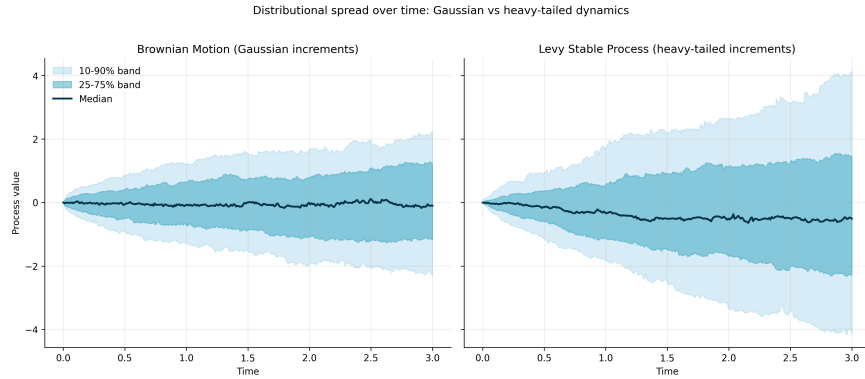


Figure 1: Quantile fan charts for Brownian and Lévy-stable ensembles. The heavy-tailed model shows broader and less regular spread dynamics over the same horizon.

Figure 1 shows why Gaussian assumptions can understate dispersion in heavy-tailed settings: quantile bands widen asymmetrically and nonlinearly.

Multiplicative heavy-tailed growth diagnostics

Figure 2 demonstrates finite-sample variability of estimated growth rates under multiplicative heavy-tailed dynamics. This is directly relevant when practitioners compare expected outcomes to path-typical behavior.

Memory-dependent process dynamics

Figure 3 illustrates how adaptive-rate memory effects produce heterogeneous mean-reverting trajectories across realizations.

Stochastic field simulation support

Figure 4 shows that the package is not limited to scalar path simulation; it also supports stochastic field-style diagnostics in space-time settings.

The project is accompanied by a long-form book with worked examples and code context (Kendiukhov 2024). At the same time, maturity constraints are explicit: selected API surfaces remain incomplete, automated pytest-style coverage is

Growth Rate vs Time and Number of Instances for GeometricLevyProcess with parameters {'alpha': 1.5, 'beta': 0, 'loc': 0.01, 'scale': 0.1}

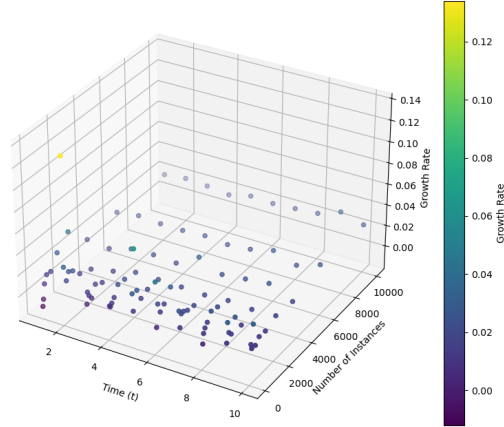


Figure 2: Geometric Lévy growth-rate map as a function of time horizon and number of instances.

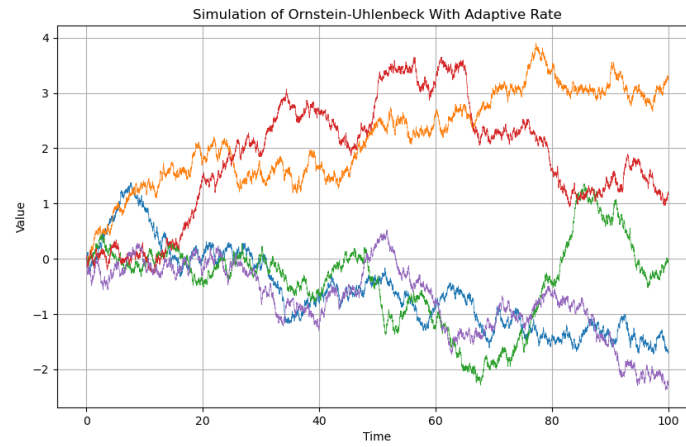


Figure 3: Adaptive-rate Ornstein–Uhlenbeck trajectories (book-derived example output).

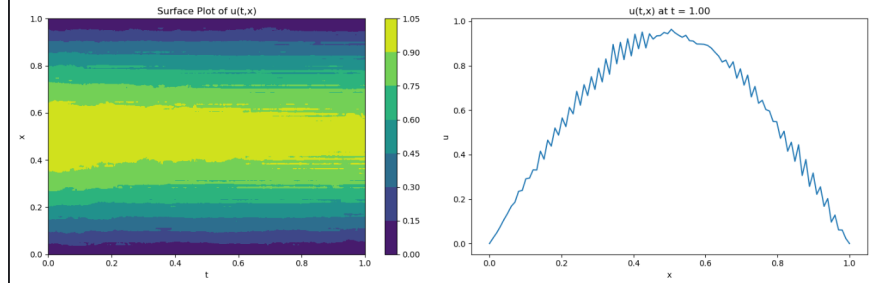


Figure 4: SPDE visualization with space-time contour (left) and final-time slice (right), from documented examples.

currently sparse relative to breadth, and optional ML-heavy agent paths can be environment-sensitive.

Research impact statement

The current impact is primarily infrastructural and methodological. `ergodicity_library` reduces setup time for experiments that combine stochastic simulation, diagnostics, and decision logic in one workflow (Kendiukhov 2025). The package already enables reproducible demonstrations for heavy-tailed spread behavior, multiplicative growth diagnostics, adaptive-memory dynamics, and stochastic field visualization (Kendiukhov 2024).

At this stage, the strongest evidence is implemented breadth plus reproducible artifacts rather than large-scale adoption metrics. The software is publicly available under an OSI-approved license and can serve as a base for domain-specific extensions.

A realistic near-term impact path is improved reproducibility and faster iteration in cross-method stochastic studies, where users can move from hypothesis definition to comparative diagnostics without rebuilding experiment scaffolding repeatedly.

AI usage disclosure

Generative AI assistance was used in preparing JOSS submission materials (manuscript drafting/editing, submission packaging, and figure-selection workflow scripting). The tools used were GPT-5-class coding assistants in an interactive development workflow.

All AI-assisted outputs were reviewed, edited, and validated by the human author. The author made the core scientific and software decisions and remains responsible for accuracy, originality, licensing compliance, and final submitted

content.

Acknowledgements

No external funding is declared for this submission. The author acknowledges the open scientific Python ecosystem that this project builds upon (Harris et al. 2020; Virtanen et al. 2020; Meurer et al. 2017; Hunter 2007).

References

- Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. “Array Programming with NumPy.” *Nature* 585: 357–62. <https://doi.org/10.1038/s41586-020-2649-2>.
- Hunter, John D. 2007. “Matplotlib: A 2D Graphics Environment.” *Computing in Science & Engineering* 9 (3): 90–95. <https://doi.org/10.1109/MCSE.2007.55>.
- Kendiukhov, Ihor. 2024. *Ergodicity Library: An Introduction*. Self-published.
- . 2025. “Ergodicity_library.” GitHub repository. https://github.com/Kendiukhov/ergodicity_library/.
- Meurer, Aaron, Christopher P. Smith, Mateusz Paprocki, et al. 2017. “SymPy: Symbolic Computing in Python.” *PeerJ Computer Science* 3: e103. <https://doi.org/10.7717/peerj-cs.103>.
- Peters, Ole. 2019. “The Ergodicity Problem in Economics.” *Nature Physics* 15: 1216–21. <https://doi.org/10.1038/s41567-019-0732-0>.
- Peters, Ole, and Murray Gell-Mann. 2016. “Evaluating Gambles Using Dynamics.” *Chaos: An Interdisciplinary Journal of Nonlinear Science* 26 (2): 023103. <https://doi.org/10.1063/1.4940236>.
- sdeint developers. 2026. “Sdeint: Numerical Integration of Stochastic Differential Equations (SDE).” PyPI project page. <https://pypi.org/project/sdeint/>.
- sdepy developers. 2026. “Sdepy: Numerical Integration of Itô Stochastic Differential Equations.” PyPI project page. <https://pypi.org/project/sdepy/>.
- stochastic developers. 2026. “Stochastic: Generate Realizations of Stochastic Processes.” PyPI project page. <https://pypi.org/project/stochastic/>.
- Virtanen, Pauli, Ralf Gommers, Travis E. Oliphant, et al. 2020. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python.” *Nature Methods* 17: 261–72. <https://doi.org/10.1038/s41592-019-0686-2>.